

VR-TAK Plugin Developer Guide

5.5.0

2025-08-11
Setup and overview



2025-08-11

Table of Contents

VR-TAK Plugin Developer Guide 5.5.0.....	1
Setup and overview	1
Overview.....	3
Unreal Editor Basics	3
Unreal Plugin Overview	3
Native / C++	4
Blueprints.....	5
Branch, sequence, cast.	6
Functions vs. Events	7
Some helpful hotkeys	8
Environment Setup.....	8
Creating a New Plugin	9

Overview

VR-TAK is made in the Unreal Engine (UE) and currently uses UE 5.3.2. This manual will give you a brief overview of how to use the Unreal Editor to set up a VR-TAK Plugin. Full Unreal documentation is online at <https://docs.unrealengine.com/>

UE is an open-source commercial game engine built in C++ by Epic Games. VR-TAK is a project built for UE 5.3.2. Developers can obtain UE 5.3.2 from Epic's Github page (<https://github.com/epicgames>), then apply the VR-TAK patch. The Engine code and VR-TAK source code is available through the TAK Product Center's Github page (<https://github.com/TAK-Product-Center/VR>)

Unreal Editor Basics

The following is a brief explanation of some core features in Unreal Engine.

When you open the editor, you are presented with a large viewport into the virtual world. Above this viewport is a green "Play" button which allows you to run a development version of VR-TAK. Below the viewport is the Content Drawer, which you can open using Space + Ctrl to see project assets.

Unreal Plugin Overview

Documentation can be found here which gives a basic overview of Plugins in Unreal Engine: <https://docs.unrealengine.com/5.0/en-US/plugins-in-unreal-engine/>
VR-TAK Plugins are dependent on VR-TAK Core, which depends on the Engine.

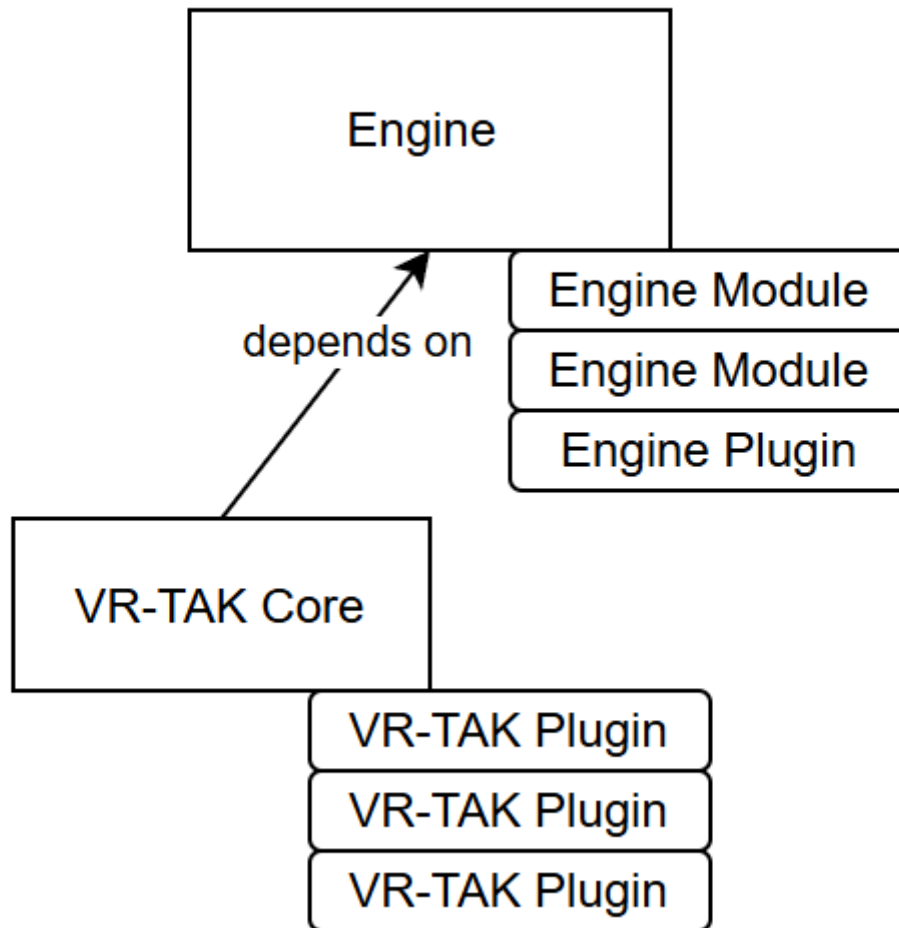


Figure 1: Engine and VR-TAK dependencies

Native / C++

As a plugin developer, you'll use the Unreal Editor and optionally C++ (also known as native).

https://dev.epicgames.com/documentation/en-us/unreal-engine/programming-with-cplusplus-in-unreal-engine?application_version=5.3

https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-programming-and-scripting?application_version=5.3

Plugins are primarily developed in Game/Plugins/<PluginNameFolder>/

Blueprints

A Blueprint (BP) is an object made and editable in the Unreal Editor. Blueprints use a visual scripting language in which you can place nodes representing functions and variables on a node graph to control the behavior of an object.

Open a blueprint using the content browser to view variables and functions. Variables and functions are listed on the left side of a blueprint, and under “Class Defaults”

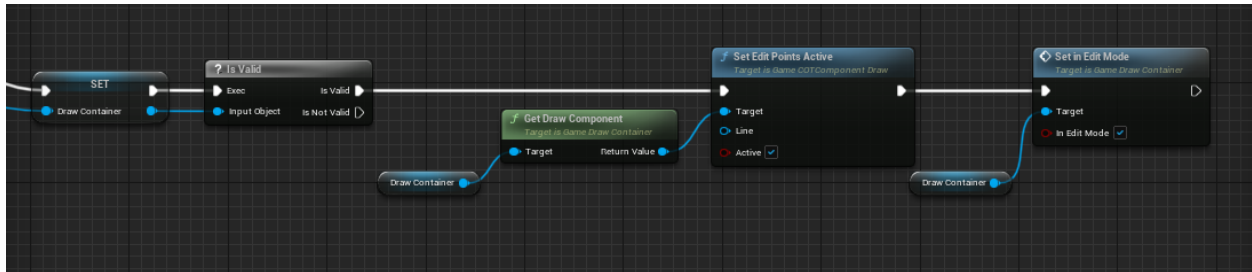


Figure 2: Blueprint nodes and execution order

Nodes are connected using wires by their pins. They should be placed such that logic goes left to right, and sometimes top to bottom.

The white wire is the execution wire, it determines the flow of logic. It starts at some origin node and continues calling all functions in order until it either ends or hits a Return node.

Events are similar to functions and are displayed on the Event Graph. Functions are each displayed in their own graph and will open when double-clicked.

Variables can be accessed using Get and Set nodes. Get nodes do not have Execution pins and should be placed close to where they are used.

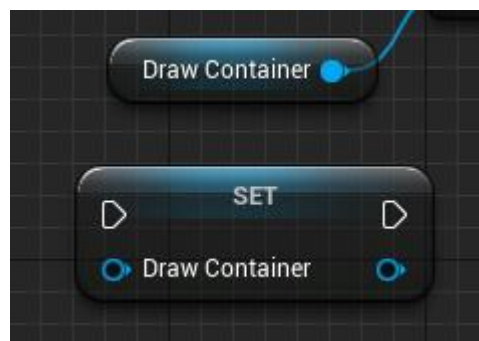


Figure 3: Variables in blueprint

Like in C++, variables can be defined on the class or locally within a function. Variables local to a function cannot be used outside of that function.

Colors on pins denote data types. Some common data types:

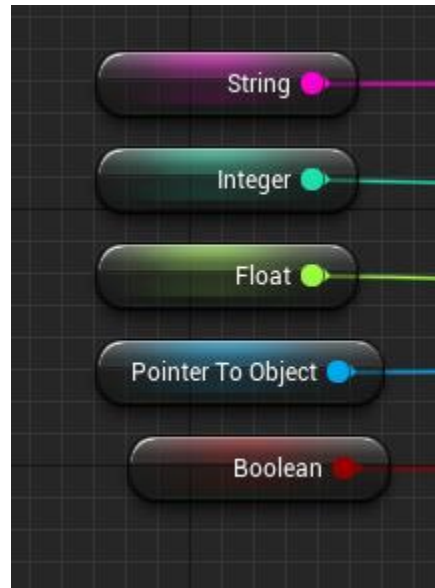


Figure 4: Blueprint variable types

Branch, sequence, cast.

A Branch is like an if statement; it has two outgoing execution nodes and will send the flow of logic towards the one that matches the connected boolean's value.

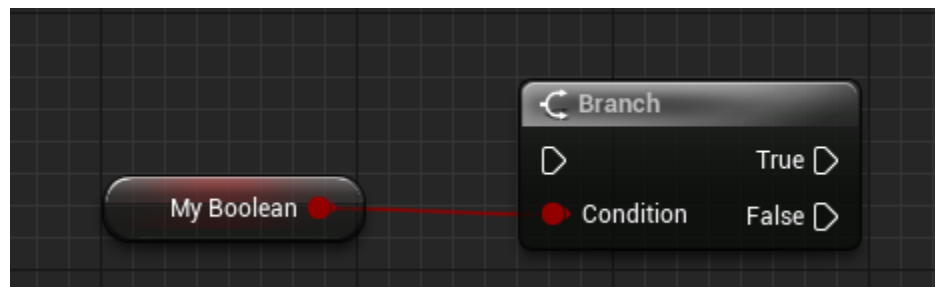


Figure 5: Blueprint branch node

A sequence will run all outgoing execution wires, one after another.

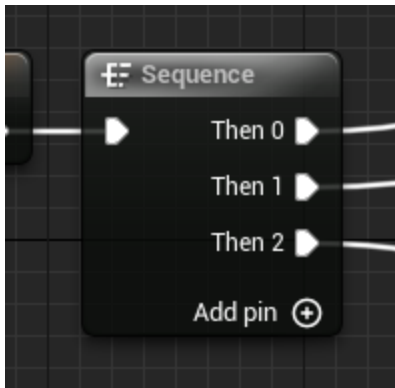


Figure 6: Blueprint sequence node

Cast nodes behave about the same as Casting in C++. In blueprint, you must cast your references to the appropriate class before you can call class-specific functions on them.

Additionally, be aware that Object pointers can be null. If there is ever a chance an object pointer is invalid, run the “Is Valid” function on it before using it.

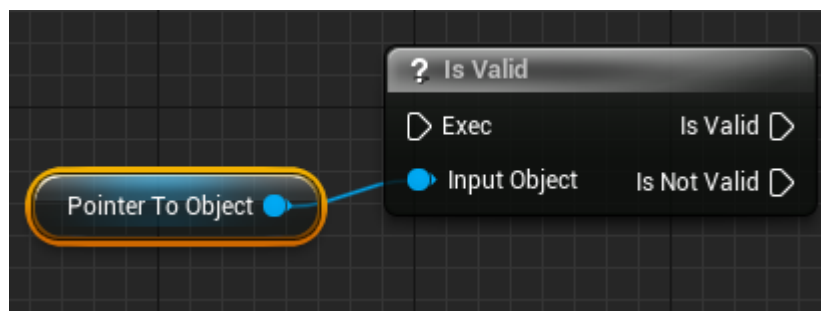


Figure 7: Blueprint IsValid node

Functions vs. Events

After opening a blueprint, a list of graph Events and Functions is shown on the left side. Double click any function or event to navigate to it

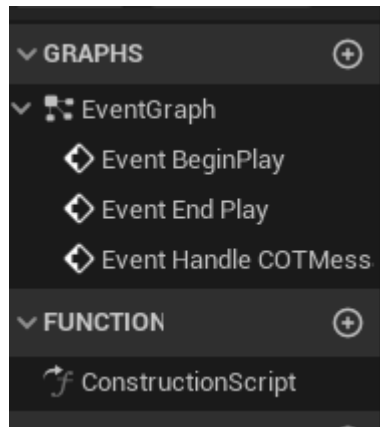


Figure 8: Blueprint graphs

Functions and events are very similar in that both handle flows of logic, similar to C++ functions.

Only functions can have return values and local variables.

Only events can have Timelines, Delay nodes (which can make things happen after a certain amount of time has passed), and support for online replication.

Some helpful hotkeys

Ctrl + S = Save the current level or blueprint (just the one on screen right now)

Ctrl + Shift + S = Save all open blueprints

Ctrl + F = Search the current blueprint

Ctrl + Shift + F = Search all blueprints

Ctrl + P = "Play" / run VR-TAK in the Unreal Editor Play-In-Editor mode.

Ctrl + Tab = Quickly Navigate between open Blueprints

After using the left mouse button to select a group of nodes in the blueprint graph editor:

C = Create a comment around the selected nodes

W = Straighten the connections between selected nodes

Double Click on an execution wire = Add a reroute node at the mouse position

To remove a wire from a node:

Hold Alt and click where the wire is connected to a node

To move a wire from a node:

Hold Ctrl and click where the wire is connected to a node

Environment Setup

1. Install Visual Studio 2022
 - a. <https://docs.unrealengine.com/en-US/Programming/Development/VisualStudioSetup/index.html>

2. Download VR-TAK from the TAK Product Center's Github
 - a. <https://github.com/TAK-Product-Center/VR>
3. Follow instructions from the readme.md in VR-TAK download

Creating a New Plugin

New plugins can be created following Epic's instructions:

https://dev.epicgames.com/documentation/en-us/unreal-engine/plugins-in-unreal-engine?application_version=5.3

Another option is to use a copy of the existing VTAK_Example plugin. This plugin adds a tool, vehicle, and new UIs accessible through the Toolbar. BP_Example_PluginActor is created at initialization and contains the blueprint code necessary for adding a new vehicle type and adding buttons to the Toolbar.

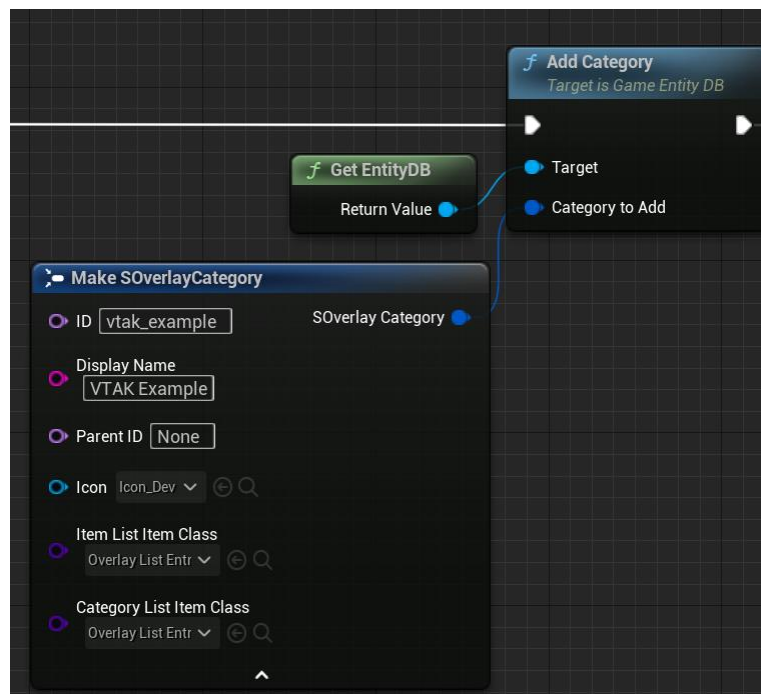


Figure 9: Adding category for Overlay Manager and other UIs to use

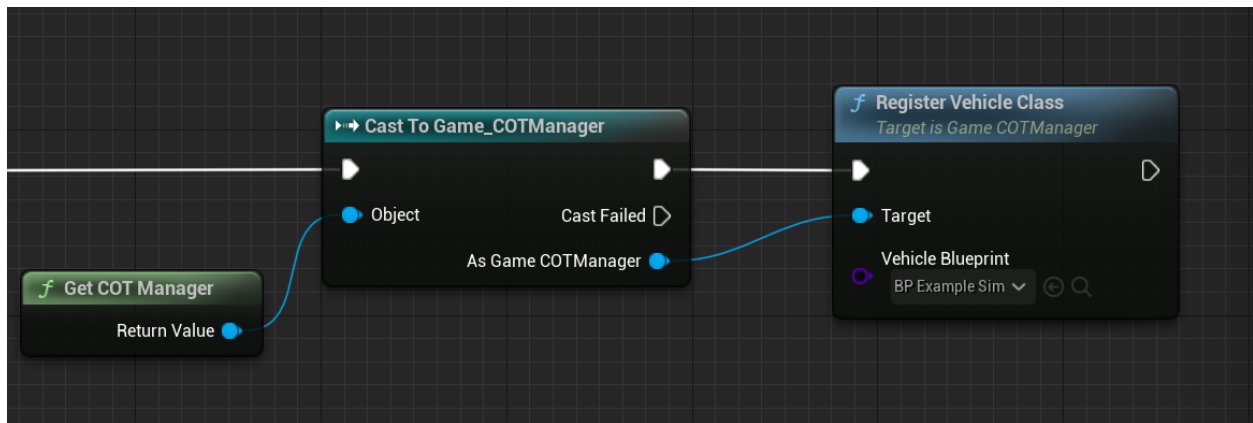


Figure 10: Adding new vehicle class for vehicle dropper

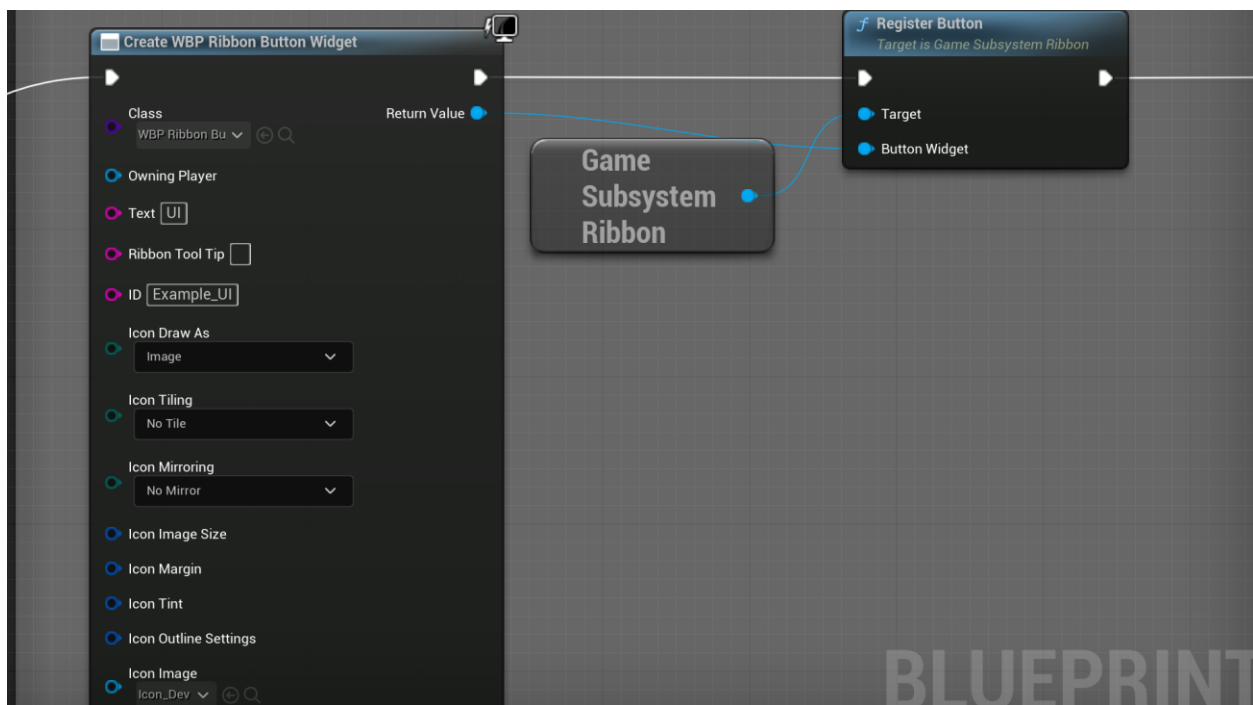


Figure 11: Adding plugin button to Toolbar

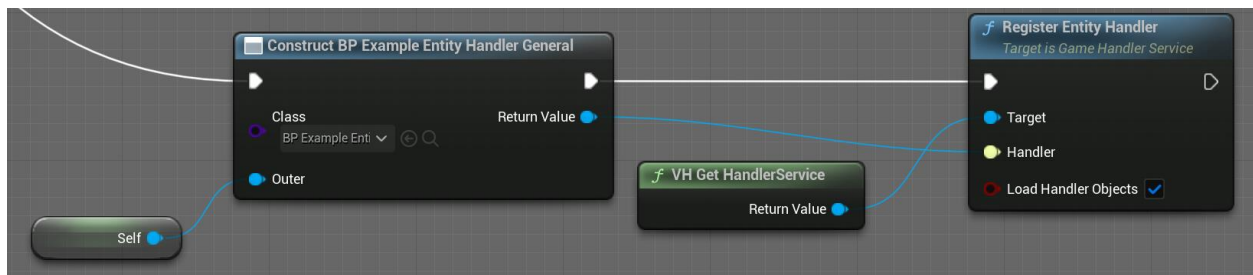


Figure 12: Adding plugin entity handler